Benchmarking Rule-Based, Classical Machine Learning, and Deep Learning Anomaly Detectors in ETL Data Quality Pipelines

Theo Bigand
ESILV — M1 Data & Artificial Intelligence
Léonard de Vinci Pôle Universitaire
Courbevoie, France
theobigand@gmail.com

Abstract*—Modern data pipelines increasingly depend on automated data quality controls, yet Extract–Transform–Load (ETL) practitioners still rely mostly on hand-crafted declarative rules. Recent work has started to introduce machine learning (ML) anomaly detectors inside ETL workflows, but systematic, apples-to-apples comparisons between rule-based, classical ML, and deep learning approaches remain scarce—especially when evaluated within a realistic pipeline and broken down by the type of anomaly being injected. This paper presents an open-source ETL pipeline in which anomaly detectors from different families share one `fit/predict/score` interface and are scored on the same pipeline path: statistical rules (Z-score, IQR, Great Expectations) and classical ML detectors (Isolation Forest, LOF, One-Class SVM), with deep learning detectors (Autoencoder, LSTM-Autoencoder) planned behind the same interface. We evaluate the detectors, over five seeds on a stratified held-out split, on the Kaggle Credit Card Fraud dataset, a Water Quality dataset, and a controlled synthetic dataset generated with Faker in which six distinct anomaly types are injected at controlled rates. Beyond aggregate precision, recall, F1, ROC-AUC and PR-AUC, we report a per-anomaly-type recall and precision that map each method to the anomaly categories it actually catches. The aggregate metrics make the families look interchangeable; the per-type view shows each has a narrow, largely non-overlapping competence, so broad coverage needs several families rather than the single best one. Our goal is a reproducible, pipeline-integrated benchmark that answers the practical question: *which family of detectors should I use, for which kind of data quality problem, at what operational cost?

***Index Terms*—data quality, ETL, anomaly detection, benchmarking, machine learning, Great Expectations, Isolation Forest, autoencoder**

I. Introduction

Extract–Transform–Load (ETL) pipelines are the backbone of modern data infrastructure: they move raw operational data into analytical stores where it feeds dashboards, reports, and machine-learning applications [1]. The value delivered downstream is bounded by the quality of the data that reaches the destination, which is why data quality has long been a first-class concern for data engineers. Poor data quality has been estimated to cost organizations an average of $12.9M per year [2], making automated, continuous quality control a critical capability rather than an optional one.

The dominant approach in industry remains rule-based: practitioners hand-write declarative expectations—"column `amount` must be positive", "column `email` must match a regex"—in frameworks such as Great Expectations, Deequ, or Soda [3]. These rules are auditable and cheap to run, but they scale poorly with schema complexity, detect only violations the author anticipated, and miss subtle multi-column or distributional anomalies. To address these limits, a recent line of work has proposed to complement or replace such rules with machine learning anomaly detectors embedded directly inside the ETL flow [2], [4], [5]. The existing literature, however, suffers from three gaps.

First, most published pipelines are conceptual architectures [2], [4] that do not ship with reproducible code or quantitative comparisons. When experiments are reported, they typically exercise a single algorithm on a single dataset—for instance, Isolation Forest on Credit Card Fraud [5], which achieved a near-random ROC-AUC of 0.51 and thus offers limited insight into when ML-based quality control is actually worth adopting.

Second, the anomaly detection community has produced excellent generic benchmarks—ADBench [6] compares 30 algorithms on 57 tabular datasets; PyOD [7] provides a unified implementation of dozens of detectors—but these benchmarks evaluate algorithms *in vacuum*, outside of an ETL context. They do not consider the declarative rule-based baselines that dominate production systems, and they do not measure end-to-end pipeline integration costs (latency, memory, operational overhead).

Third, benchmarks typically report a single aggregated score per dataset, hiding the fact that different families of detectors excel at different kinds of anomalies. A Z-score detector is essentially unable to flag categorical format violations; Great Expectations trivially catches null-rate spikes but ignores multivariate outliers [8]; autoencoders learn joint distributions but can be fooled by concept drift. A practitioner choosing

a detector today lacks a fine-grained, actionable map from anomaly type to recommended method.

a) Contributions.: This paper addresses these three gaps. We present:

1) **A reproducible ETL pipeline** in which rule-based and classical ML anomaly detectors share one interface and are scored on the same extract–transform–detect path, released as open-source code, with deep-learning detectors planned behind the same interface.
2) **A head-to-head benchmark** of statistical rules (Z-score, IQR, Great Expectations) and classical ML detectors (Isolation Forest, LOF, One-Class SVM) on three datasets, with a stratified held-out split, five seeds, and the ML operating point read from the data, reporting precision, recall, F1, ROC-AUC, PR-AUC, and in-pipeline latency.
3) **A per-anomaly-type analysis.** Using a synthetic dataset in which six distinct anomaly categories (missing values, duplicates, univariate outliers, multivariate outliers, temporal drift, invalid formats) are injected at controlled rates, we produce a method-by-type detection-rate matrix. The families turn out to have narrow, largely non-overlapping competence that the aggregate metrics hide.

The rest of the paper is organized as follows. Section II reviews related work on data quality in ETL and on anomaly detection benchmarks. Section III describes our pipeline architecture, the detectors we compare, and the datasets and metrics we use. Section IV reports experimental results. Section V discusses when rules, classical ML, and deep learning are each the right tool, and highlights practical trade-offs in latency and operational cost. Section VI concludes and outlines future work.

II. Related Work

To be written. Planned subsections:

- Data quality in ETL pipelines (Khatri & Brown [1]; Data Quality Survey [9])
- Rule-based and declarative approaches (Great Expectations, Deequ, Soda [3])
- Classical ML for anomaly detection (Isolation Forest, LOF, One-Class SVM; Goldstein & Uchida [8]; PyOD [7])
- Deep learning (Autoencoder, LSTM-Autoencoder)
- Existing benchmarks: ADBench [6] and why it does not cover rule-based baselines, ETL integration, or per-anomaly-type analysis
- ML in ETL pipelines: Gopa [2], Marupaka & Rangineni [4], Sokkula & Vuppala [5] as illustrative limitations

III. Methodology

A. Pipeline

Detectors are evaluated inside a minimal Extract–Transform–Load pipeline rather than in isolation. The extract stage reads the raw table, the transform stage removes exact duplicate rows, and the quality stage applies a detector that assigns each row a boolean anomaly flag and a continuous anomaly score. All detectors share a common `fit`/`predict`/`score` interface modelled on PyOD [7], so that classical and deep models can later be dropped in without changing the harness.

B. Rule-based detectors

This stage compares three rule-based baselines. All three define a band of "normal" values per column and flag a row when it leaves the band on at least one column; they differ in how the band is estimated and in how robust that estimate is to the very outliers being sought.

Z-score standardizes each numeric column on the fitted data and flags a row when any column exceeds $|z| > 3$. The threshold follows the empirical rule: under an approximately normal distribution about 99.7% of values lie within 3σ, so beyond it is rare by construction. Its weakness is that the mean and standard deviation are themselves inflated by extreme values, so a large outlier can mask others.

IQR learns Tukey's fences $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$ per column, where the interquartile range spans the central 50% of the data, and flags a row falling outside any fence. Quartiles are robust: unlike the mean and standard deviation they barely move when extreme values are added, so the fences are not distorted by the outliers they are meant to catch. The 1.5 multiplier corresponds to about 2.7σ under normality, isolating a comparable tail.

Great Expectations encodes a declarative expectation suite [3] and makes no distributional assumption: a not-null expectation on every column and a range expectation per numeric column, with bounds set to the 0.1th and 99.9th percentiles of the fitted data. Using inner percentiles rather than the observed min/max is what turns the range check into a usable rule instead of a no-op on the fitting data. A row is flagged when it violates at least one expectation; its score is the fraction of the suite it violates.

The continuous scores of the statistical detectors are the worst standardized deviation (Z-score) and the largest fence exceedance in IQR units (IQR). The axis of difference between the three is thus robustness and the assumption made about the shape of the data, which is precisely what the learned models of the next stage are meant to handle better.

C. Classical machine-learning detectors

The second family replaces the per-column band with a learned model of the joint feature space, so that a row can be flagged for an implausible *combination* of otherwise-normal values. Three unsupervised detectors are taken from PyOD [7] rather than reimplemented: *Isolation Forest*, which scores a row by how few random axis-aligned splits are needed to isolate it; *Local Outlier Factor* (LOF), which compares a point's local density to that of its neighbours; and a *One-Class SVM*, which fits a boundary enclosing the bulk of the training data. All three receive the same preparation: non-numeric columns are dropped, missing cells are median-imputed, and every feature is standardized, since LOF and the One-Class SVM are distance-based. The PyOD `decision_function` provides

the continuous score; the boolean flag uses a *contamination* equal to the anomaly prevalence of the training split (clamped to PyOD's valid range), so each model marks about as many rows as the data actually contains rather than a fixed share. Contamination sets only the flag threshold and leaves ROC-AUC and PR-AUC unaffected.

D. Synthetic per-anomaly-type dataset

The real datasets carry a single binary label that mixes every kind of defect together, so they cannot say *which* problem a detector is good at. We therefore generate a controlled dataset of 12,360 customer records with Faker and NumPy, in which income is correlated with balance and transaction size and account age is spread over three years. Into this clean population we inject six anomaly types, each at a rate of 3% and each row tagged with its category: *missing values* (one or two cells nulled), *exact duplicates* (a clean row copied), *univariate outliers* (one numeric field pushed far out of range), *multivariate outliers* (every field individually in range but the combination implausible, e.g. an 18-year-old with a top-percentile balance and income), *temporal drift* (recent-signup rows whose income and balance distribution has shifted upward by half), and *invalid formats* (malformed e-mail, out-of-domain country code, impossible age). Because the category of every injected row is known, we report, per method and per type, both the detection rate (recall) and the precision on a stream made of clean rows plus that one type.

E. Datasets

We evaluate on three datasets. *Credit Card Fraud* (Kaggle; 283,726 transactions after de-duplication, 30 numeric features, 0.17% labelled fraud) is the standard strongly imbalanced benchmark. The *synthetic* dataset above (12,360 rows, 17.5% anomalous) carries the per-type labels. *Water Quality* (a mirror of the UCI-style water-potability data; 3,276 samples, 9 numeric features) is a third real dataset in the thesis's application domain; here the target class ("not potable") is common rather than rare (61%) and the data has genuine missing values, so it probes the detectors outside the regime they are designed for.

F. Evaluation protocol

Each dataset is split $70/15/15$ (train/validation/test), stratified on the label. Detectors are fitted on the train split and every reported number is computed on the held-out test split; the validation split is reserved for later tuning. The ML contamination is read from the train split's prevalence. The whole procedure is repeated over five random seeds and we report the mean and standard deviation. LOF and the One-Class SVM are run on Water Quality and the synthetic dataset but not on Credit Card Fraud, where LOF's superlinear scoring and the One-Class SVM's quadratic training do not fit inside the pipeline at 284k rows.

Because the detectors emit both a threshold decision and a graded score, we report threshold metrics (precision, recall, F_1) alongside ROC-AUC and PR-AUC. On imbalanced data PR-AUC is the more informative metric: the false-positive rate underlying ROC-AUC is diluted by the large true-negative pool and stays optimistic, whereas precision reacts directly to false alarms. We keep both because PR-AUC's random baseline equals the prevalence and is not comparable across datasets, while ROC-AUC's is always 0.5 and is. We also report the fraction of rows flagged and the in-pipeline latency. The deep-learning detectors (Autoencoder, LSTM-Autoencoder) are the object of the next stage.

TABLE I
DETECTORS ON CREDIT CARD FRAUD, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. HIGHER IS BETTER EXCEPT FLAGGED ROWS AND TIME.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%	Time (s)
Z-score	0.011	0.870	0.021	0.926	0.078	13.4	**0.03**
IQR	0.003	**0.966**	0.007	0.911	0.018	48.6	0.12
Great Expectations	0.037	0.676	**0.070**	0.830	**0.182**	3.1	1.37
Isolation Forest	**0.202**	0.206	**0.203**	**0.934**	0.120	**0.2**	1.78

IV. EXPERIMENTS AND RESULTS

All numbers below are the mean over five seeds on the held-out test split (Section III-F); standard deviations are ≤ 0.05 unless the text says otherwise.

A. Real fraud data

Table I reports the detectors on Credit Card Fraud. The statistical rules recover most frauds (recall 0.87–0.97) at a precision below 0.04: a fixed per-column rule cannot separate a 0.17% positive class from the tails of legitimate traffic. Great Expectations is the most selective (3% of rows) and leads PR-AUC at 0.182 ± 0.046. Isolation Forest behaves differently now that its contamination matches the prevalence: it flags only 0.17% of rows and reaches a precision of 0.20, the first usable operating point in the table, but its recall drops to 0.21 and both are seed-sensitive ($F_1 = 0.20 \pm 0.07$). It keeps the best ROC-AUC (0.934). Great Expectations remains the slowest linear method (1.4 s on the test split against 0.03–0.12 s for the statistical rules). LOF and the One-Class SVM are absent: neither scales to 284k rows in the pipeline.

B. Synthetic dataset: aggregate metrics

Table II reports all six detectors on the synthetic dataset. Aggregate metrics still compress the families together: F_1 ranges from 0.31 to 0.51 and PR-AUC from 0.33 to 0.56. Isolation Forest now has the best F_1 (0.51) and Z-score keeps a perfect precision by firing only on the most extreme rows. The One-Class SVM matches Isolation Forest on quality at about $16\times$ the latency. Read alone, these numbers suggest the detector choice barely matters; the per-type breakdown shows the opposite.

C. Synthetic dataset: by anomaly type

Fig. 1 gives the mean recall of each method on each injected type. The structure the aggregate table hides is now clear: every method has a narrow competence.

TABLE II
ALL DETECTORS ON THE SYNTHETIC DATASET, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TIME IS THE IN-PIPELINE FIT+PREDICT+SCORE LATENCY.

Detector	Prec.	Rec.	F_1	ROC	PR	Time (s)
Z-score	**1.000**	0.182	0.308	**0.748**	**0.560**	**0.002**
IQR	0.603	0.399	0.481	0.678	0.460	0.004
Great Expectations	0.920	0.205	0.335	0.601	0.332	0.29
Isolation Forest	0.515	**0.503**	**0.509**	0.745	0.501	0.16
LOF	0.380	0.390	0.385	0.646	0.437	0.14
One-Class SVM	0.488	0.478	0.483	0.729	0.531	2.61

- *Univariate outliers* are caught by every distributional method (recall 0.92–0.99) and only by those. Z-score's precision on them is 1.00: it never fires on a clean row.
- *Missing values* are caught only by the Great Expectations not-null expectations (recall 1.00, precision 0.90), because the statistical rules propagate `NaN` to a non-flag and the learned models see imputed values.
- *Exact duplicates* are invisible to every detector. The deduplication rule of the transform stage is the only thing that catches them, at precision 0.52 because it also flags each row's clean original.
- *Multivariate outliers* separate the families: Isolation Forest (0.91) and the One-Class SVM (0.84) are the only methods with real recall, since they are the only ones that model the joint distribution. Their precision on this type is only about 0.24: they cast a wide net that also catches clean rows.
- *Invalid formats* are caught mainly through the impossible numeric value they carry (IQR recall 1.00, precision 0.39; Z-score precision 1.00); the malformed string fields are missed by all.
- *Temporal drift* is the weakest column for every method (best recall 0.49, LOF), because the shifted values stay inside the global range and no detector uses the timestamp.

The practical reading is a short decision list: null-rate problems to declarative rules, single-column outliers to any statistical rule, implausible combinations to a tree- or boundary-based model, and duplicates and drift to dedicated stages the detectors do not replace.

D. Third real dataset: Water Quality

Table III runs all six detectors on Water Quality, where "not potable" is the target and covers 61% of the rows. Every detector's ROC-AUC sits between 0.45 and 0.51 and every PR-AUC is at or just above the 0.61 base rate: the unsupervised detectors are essentially at chance. This is the expected outcome and a useful negative result. When the target class is common and not distributionally distinct from the rest, "anomaly" detection has nothing to key on. The dataset still exercises the pipeline on real missing data: Great Expectations flags 40% of rows on its not-null expectations, but the nulls here are uncorrelated with potability, so that recall buys no precision.

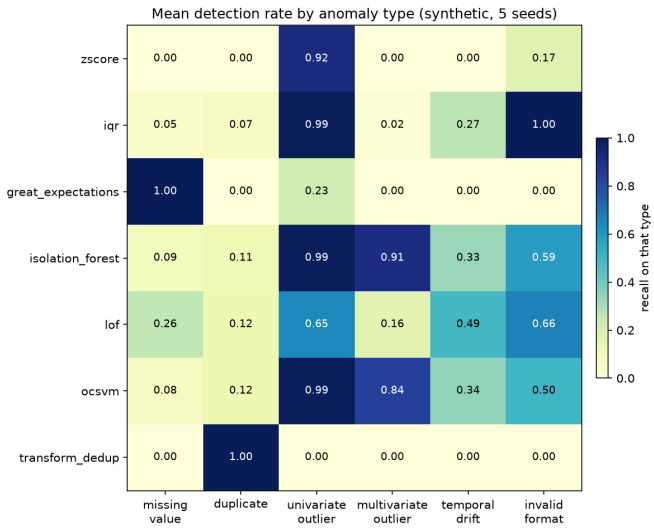


Fig. 1. Mean detection rate (recall) over five seeds of each method on each injected anomaly type, synthetic test split. Darker is higher. `transform_dedup` is the deduplication rule of the transform stage, shown for reference.

TABLE III
ALL DETECTORS ON WATER QUALITY, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TARGET: "NOT POTABLE", BASE RATE 0.61.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%
Z-score	0.41	0.03	0.05	0.46	0.58	3.5
IQR	0.57	0.09	0.16	0.49	0.60	10.0
Great Expectations	0.62	0.41	0.49	0.51	0.62	40.1
Isolation Forest	0.57	0.47	0.51	0.45	0.58	49.7
LOF	0.58	0.46	0.52	0.46	0.58	48.7
One-Class SVM	0.57	0.45	0.50	0.46	0.57	48.4

V. DISCUSSION

a) Rules and learned models are complementary, not competing.: The per-type heatmap shows almost no overlap between what the declarative rules catch (missing values), what the statistical rules catch (single-column outliers), and what the learned models add (implausible combinations, recall 0.91 for Isolation Forest against ≤ 0.02 for the rules). Broad coverage needs more than one family, not the single best one on aggregate. This is a different conclusion from benchmarks that publish a single ranking.

b) The operating point matters as much as the algorithm.: Setting Isolation Forest's contamination to the data prevalence rather than a fixed 0.10 moves its fraud precision from 0.01 to 0.20 and its multivariate-outlier recall on the synthetic data from 0.41 to 0.91. The same estimator looks weak or strong depending on one number that a benchmark must choose deliberately.

c) Aggregate metrics on a mixed label are misleading.: On all three datasets the aggregate table compresses different behaviour into similar F_1 and PR-AUC values. A method excellent on one anomaly type and blind to the rest scores like a mediocre all-rounder. The controlled per-type dataset is

what makes the difference measurable.

d) Pipeline integration changes the shortlist.: Latency and scaling are not secondary. Great Expectations is the slowest linear method; the One-Class SVM's quadratic training and LOF's superlinear scoring keep both off the full fraud dataset. A benchmark scoring algorithms in isolation would not surface this.

e) Threats to validity.: The synthetic dataset encodes our own assumptions about what each anomaly type looks like, and the injection magnitudes are choices. Water Quality shows the methods only outside their regime; a genuinely imbalanced second real dataset is still needed before the per-type ranking can be called general. Isolation Forest's seed variance on fraud ($F_1 = 0.20 \pm 0.07$) means its precision gain there should be read as a range, not a point.

VI. Conclusion

We built an ETL pipeline in which rule-based and classical machine-learning anomaly detectors share one interface and are scored on the same held-out path, over five seeds, with the operating point read from the data. A controlled dataset labels six anomaly types separately so detection can be measured per type. On aggregate metrics the families stay close on all three datasets. The per-type view is where they diverge: declarative rules own missing values, statistical rules own single-column outliers, tree- and boundary-based models are the only ones that catch implausible combinations, and duplicates and temporal drift need dedicated stages. The choice of contamination and the pipeline cost further constrain which detector fits a real flow.

The next steps are the deep-learning detectors (Autoencoder, then LSTM-Autoencoder) behind the same interface, a genuinely imbalanced second real dataset, and a detector that uses the timestamp so temporal drift stops being an empty column.

References

[1] V. Khatri and C. V. Brown, "Designing data governance," *Communications of the ACM*, vol. 53, no. 1, pp. 148–152, 2010.

[2] R. Gopa, "AI-augmented ETL pipelines for automated data quality, anomaly detection and governance," *International Journal of Computational and Experimental Science and Engineering (IJCESEN)*, vol. 11, no. 4, 2025.

[3] Various Authors, "A survey of pipeline tools for data engineering," *arXiv preprint arXiv:2406.08335*, 2024. [Online]. Available: https://arxiv.org/abs/2406.08335

[4] P. Marupaka and S. Rangineni, "Machine learning-driven predictive data quality assessment in ETL frameworks," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 72, no. 3, 2024.

[5] S. R. Sokkula and S. R. Vuppala, "Implementing machine learning for ETL data transformation and anomaly detection," *International Journal for Multidisciplinary Research (IJFMR)*, vol. 6, no. 6, 2024.

[6] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://openreview.net/pdf?id=foA_SFQ9zo0

[7] Y. Zhao, Z. Nasrullah, and Z. Li, "PyOD: A python toolbox for scalable outlier detection," *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 2019.

[8] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLoS ONE*, vol. 11, no. 4, p. e0152173, 2016.

[9] Various Authors, "A survey on data quality dimensions and tools for machine learning," *arXiv preprint arXiv:2406.19614*, 2024. [Online]. Available: https://arxiv.org/abs/2406.19614